New York City Yellow Taxi Data

Objective

In this case study you will be learning exploratory data analysis (EDA) with the help of a dataset on yellow taxi rides in New York City. This will enable you to understand why EDA is an important step in the process of data science and machine learning.

Problem Statement

As an analyst at an upcoming taxi operation in NYC, you are tasked to use the 2023 taxi trip data to uncover insights that could help optimise taxi operations. The goal is to analyse patterns in the data that can inform strategic decisions to improve service efficiency, maximise revenue, and enhance passenger experience.

Tasks

You need to perform the following steps for successfully completing this assignment:

1. Data Loading
2. Data Cleaning
3. Exploratory Analysis: Bivariate and Multivariate
4. Creating Visualisations to Support the Analysis
5. Deriving Insights and Stating Conclusions

NOTE: The marks given along with headings and sub-headings are cumulative marks for those particular headings/sub-headings.

The actual marks for each task are specified within the tasks themselves.

For example, marks given with heading *2* or sub-heading *2.1* are the cumulative marks, for your reference only.

The marks you will receive for completing tasks are given with the tasks.

Suppose the marks for two tasks are: 3 marks for 2.1.1 and 2 marks for 3.2.2, or

- 2.1.1 [3 marks]
- 3.2.2 [2 marks]

then, you will earn 3 marks for completing task 2.1.1 and 2 marks for completing task 3.2.2.

Data Understanding

The yellow taxi trip records include fields capturing pick-up and drop-off dates/times, pick-up and drop-off locations, trip distances, itemized fares, rate types, payment types, and driver-reported passenger counts.

The data is stored in Parquet format (*.parquet*). The dataset is from 2009 to 2024. However, for this assignment, we will only be using the data from 2023.

The data for each month is present in a different parquet file. You will get twelve files for each of the months in 2023.

The data was collected and provided to the NYC Taxi and Limousine Commission (TLC) by technology providers like vendors and taxi hailing apps.

You can find the link to the TLC trip records page here: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Data Description

You can find the data description here: Data Dictionary

Trip Records

Field Name	description
VendorID	A code indicating the TPEP provider that provided the record. 1= Creative Mobile Technologies, LLC; 2= VeriFone Inc.
tpep_pickup_datetime	The date and time when the meter was engaged.
tpep_dropoff_datetime	The date and time when the meter was disengaged.
Passenger_count	The number of passengers in the vehicle. This is a driver-entered value.
Trip_distance	The elapsed trip distance in miles reported by the taximeter.
PULocationID	TLC Taxi Zone in which the taximeter was engaged
DOLocationID	TLC Taxi Zone in which the taximeter was disengaged
RateCodeID	The final rate code in effect at the end of the trip. 1 = Standard rate 2 = JFK 3 = Newark 4 = Nassau or Westchester 5 = Negotiated fare 6 = Group ride
Store_and_fwd_flag	This flag indicates whether the trip

Field Name	description
	record was held in vehicle memory before sending to the vendor, aka “store and forward,” because the vehicle did not have a connection to the server. Y= store and forward trip N= not a store and forward trip
Payment_type	A numeric code signifying how the passenger paid for the trip. 1 = Credit card 2 = Cash 3 = No charge 4 = Dispute 5 = Unknown 6 = Voided trip
Fare_amount	The time-and-distance fare calculated by the meter. Extra Miscellaneous extras and surcharges. Currently, this only includes the 0.50 and 1 USD rush hour and overnight charges.
MTA_tax	0.50 USD MTA tax that is automatically triggered based on the metered rate in use.
Improvement_surcharge	0.30 USD improvement surcharge assessed trips at the flag drop. The improvement surcharge began being levied in 2015.
Tip_amount	Tip amount – This field is automatically populated for credit card tips. Cash tips are not included.
Tolls_amount	Total amount of all tolls paid in trip.
total_amount	The total amount charged to passengers. Does not include cash tips.
Congestion_Surcharge	Total amount collected in trip for NYS congestion surcharge.
Airport_fee	1.25 USD for pick up only at LaGuardia and John F. Kennedy Airports

Although the amounts of extra charges and taxes applied are specified in the data dictionary, you will see that some cases have different values of these charges in the actual data.

Taxi Zones

Each of the trip records contains a field corresponding to the location of the pickup or drop-off of the trip, populated by numbers ranging from 1-263.

These numbers correspond to taxi zones, which may be downloaded as a table or map/shapefile and matched to the trip records using a join.

This is covered in more detail in later sections.

---

1 Data Preparation

[5 marks]

Import Libraries

```
# Import warnings
import warnings
warnings.filterwarnings("ignore")

# Import the libraries you will be using for analysis

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Recommended versions
# numpy version: 1.26.4
# pandas version: 2.2.2
# matplotlib version: 3.10.0
# seaborn version: 0.13.2

# Check versions
print("numpy version:", np.__version__)
print("pandas version:", pd.__version__)
print("matplotlib version:", plt.matplotlib.__version__)
print("seaborn version:", sns.__version__)
```

```
numpy version: 1.26.4
pandas version: 2.2.2
matplotlib version: 3.9.2
seaborn version: 0.13.2
```

1.1 Load the dataset

[5 marks]

You will see twelve files, one for each month.

To read parquet files with Pandas, you have to follow a similar syntax as that for CSV files.

`df = pd.read_parquet('file.parquet')`

```
# Try loading one file
df = pd.
read_parquet(r'/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-1.parquet')
```

```
df.info()
df.head ( )
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 3041714 entries, 0 to 3066765
Data columns (total 19 columns):
 #   Column                 Dtype
---  ------                 -----
 0   VendorID               int64
 1   tpep_pickup_datetime   datetime64[us]
 2   tpep_dropoff_datetime  datetime64[us]
 3   passenger_count        float64
 4   trip_distance          float64
 5   RatecodeID             float64
 6   store_and_fwd_flag     object
 7   PULocationID           int64
 8   DOLocationID           int64
 9   payment_type           int64
 10  fare_amount            float64
 11  extra                  float64
 12  mta_tax                float64
 13  tip_amount             float64
 14  tolls_amount           float64
 15  improvement_surcharge  float64
 16  total_amount           float64
 17  congestion_surcharge   float64
 18  airport_fee            float64
dtypes: datetime64[us](2), float64(12), int64(4), object(1)
memory usage: 464.1+ MB
```

```
   VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
0         2  2023-01-01 00:32:10   2023-01-01 00:40:36
1.0
1         2  2023-01-01 00:55:08   2023-01-01 01:01:27
1.0
2         2  2023-01-01 00:25:04   2023-01-01 00:37:49
1.0
3         1  2023-01-01 00:03:48   2023-01-01 00:13:25
0.0
4         2  2023-01-01 00:10:29   2023-01-01 00:21:19
1.0

   trip_distance  RatecodeID store_and_fwd_flag  PULocationID
DOLocationID  \
0           0.97         1.0                  N           161
141
1           1.10         1.0                  N            43
237
2           2.51         1.0                  N            48
```

```
238
3            1.90          1.0                  N           138
7
4            1.43          1.0                  N           107
79

   payment_type  fare_amount  extra  mta_tax  tip_amount  tolls_amount
\
0             2          9.3   1.00      0.5        0.00           0.0

1             1          7.9   1.00      0.5        4.00           0.0

2             1         14.9   1.00      0.5       15.00           0.0

3             1         12.1   7.25      0.5        0.00           0.0

4             1         11.4   1.00      0.5        3.28           0.0


   improvement_surcharge  total_amount  congestion_surcharge
airport_fee
0                    1.0         14.30                   2.5
0.00
1                    1.0         16.90                   2.5
0.00
2                    1.0         34.90                   2.5
0.00
3                    1.0         20.85                   0.0
1.25
4                    1.0         19.68                   2.5
0.00
```

How many rows are there? Do you think handling such a large number of rows is computationally feasible when we have to combine the data for all twelve months into one?

To handle this, we need to sample a fraction of data from each of the files. How to go about that? Think of a way to select only some portion of the data from each month's file that accurately represents the trends.

Sampling the Data

> One way is to take a small percentage of entries for pickup in every hour of a date. So, for all the days in a month, we can iterate through the hours and select 5% values randomly from those. Use `tpep_pickup_datetime` for this. Separate date and hour from the datetime values and then for each date, select some fraction of trips for each of the 24 hours.

To sample data, you can use the `sample()` method. Follow this syntax:

```
# sampled_data is an empty DF to keep appending sampled data of each
hour
```

```
# hour_data is the DF of entries for an hour 'X' on a date 'Y'

sample = hour_data.sample(frac = 0.05, random_state = 42)
# sample 0.05 of the hour_data
# random_state is just a seed for sampling, you can define it yourself

sampled_data = pd.concat([sampled_data, sample]) # adding data for
this hour to the DF
```

This *sampled_data* will contain 5% values selected at random from each hour.

Note that the code given above is only the part that will be used for sampling and not the complete code required for sampling and combining the data files.

Keep in mind that you sample by date AND hour, not just hour. (Why?)

1.1.1 [5 marks] Figure out how to sample and combine the files.

Note: It is not mandatory to use the method specified above. While sampling, you only need to make sure that your sampled data represents the overall data of all the months accurately.

```
# Sample the data
# It is recommmended to not load all the files at once to avoid memory
overload

# from google.colab import drive
# drive.mount('/content/drive')

# Take a small percentage of entries from each hour of every date.
# Iterating through the monthly data:
#   read a month file -> day -> hour: append sampled data -> move to
next hour -> move to next day after 24 hours -> move to next month
file
# Create a single dataframe for the year combining all the monthly
data

# Select the folder having data files
import os

# Select the folder having data files
os.chdir(r'/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records')
# Create a list of all the twelve files to read
file_list = os.listdir()

# initialise an empty dataframe
df = pd.DataFrame()
```

```
# iterate through the list of files and sample one by one:
for file_name in file_list:
    try:
        # file path for the current file
        file_path = os.path.join(os.getcwd(), file_name)
        print(file_path)
        # Reading the current file
        monthly_data = pd.read_parquet(file_path)
        #Extracting the data
        monthly_data['date'] =
monthly_data['tpep_pickup_datetime'].dt.date
        monthly_data['hour'] =
monthly_data['tpep_pickup_datetime'].dt.hour

        # We will store the sampled data for the current date in this
df by appending the sampled data from each hour to this
        # After completing iteration through each date, we will append
this data to the final dataframe.
        sampled_data = pd.DataFrame()

        # Loop through dates and then loop through every hour of each
date
        for date in monthly_data['date'].unique():
            daily_data=monthly_data[monthly_data['date'] ==
date].copy()
            # Iterate through each hour of the selected date
            for hour in range(24):
                hour_data=daily_data[monthly_data['hour'] == hour]
                # Sample 5% of the hourly data randomly
                if not hour_data.empty:
                    sample = hour_data.sample(frac = 0.05,
random_state = 42)
                    # add data of this hour to the dataframe
                    sampled_data = pd.concat([sampled_data, sample])

        # Concatenate the sampled data of all the dates to a single
dataframe
        df = pd.concat([df, sampled_data])# we initialised this empty
DF earlier

    except Exception as e:
        print(f"Error reading file {file_name}: {e}")

# Reset index after combining all months
df.reset_index(drop=True, inplace=True)

# Show summary of sampled dataset
print(df.head())
```

```
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-12.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-6.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-7.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/.DS_Store
Error reading file .DS_Store: Could not open Parquet input source
'<Buffer>': Parquet magic bytes not found in footer. Either the file
is corrupted or this is not a parquet file.
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-5.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-11.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-10.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-4.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-1.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-8.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-9.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-2.parquet
/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records/2023-3.parquet
   VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
0         2  2023-12-01 00:27:51   2023-12-01 00:50:12
1.0
1         2  2023-12-01 00:38:48   2023-12-01 01:01:55
NaN
2         2  2023-12-01 00:06:19   2023-12-01 00:16:57
1.0
3         2  2023-12-01 00:00:50   2023-12-01 00:14:37
NaN
4         2  2023-12-01 00:16:07   2023-12-01 00:19:17
1.0

   trip_distance  RatecodeID store_and_fwd_flag  PULocationID
DOLocationID  \
0           3.99         1.0                  N           148
50
1           4.79         NaN               None           231
61
2           1.05         1.0                  N           161
161
```

```
3           2.08         NaN                 None            137
144
4           0.40         1.0                    N             68
68

   payment_type  ...  mta_tax  tip_amount  tolls_amount  \
0             1  ...      0.5        5.66           0.0
1             0  ...      0.5        3.00           0.0
2             1  ...      0.5        3.14           0.0
3             0  ...      0.5        0.00           0.0
4             1  ...      0.5        0.00           0.0

   improvement_surcharge  total_amount  congestion_surcharge
Airport_fee  \
0                    1.0         33.96                   2.5
0.0
1                    1.0         29.43                   NaN
NaN
2                    1.0         18.84                   2.5
0.0
3                    1.0         21.22                   NaN
NaN
4                    1.0         10.10                   2.5
0.0

         date  hour airport_fee
0  2023-12-01     0         NaN
1  2023-12-01     0         NaN
2  2023-12-01     0         NaN
3  2023-12-01     0         NaN
4  2023-12-01     0         NaN

[5 rows x 22 columns]
```

After combining the data files into one DataFrame, convert the new DataFrame to a CSV or parquet file and store it to use directly.

Ideally, you can try keeping the total entries to around 250,000 to 300,000.

```
# Store the df in csv/parquet
df.to_parquet(r'/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records.parquet',index=False)
```

Now we can load the new data directly.

```
# Load the new data file
df =
pd.read_parquet(r'/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records.parquet')
```

```
df.head()
```

```
   VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
0         2  2023-12-01 00:27:51   2023-12-01 00:50:12
1.0
1         2  2023-12-01 00:38:48   2023-12-01 01:01:55
NaN
2         2  2023-12-01 00:06:19   2023-12-01 00:16:57
1.0
3         2  2023-12-01 00:00:50   2023-12-01 00:14:37
NaN
4         2  2023-12-01 00:16:07   2023-12-01 00:19:17
1.0

   trip_distance  RatecodeID store_and_fwd_flag  PULocationID
DOLocationID  \
0           3.99         1.0                  N           148
50
1           4.79         NaN               None           231
61
2           1.05         1.0                  N           161
161
3           2.08         NaN               None           137
144
4           0.40         1.0                  N            68
68

   payment_type  ...  mta_tax  tip_amount  tolls_amount  \
0             1  ...      0.5        5.66           0.0
1             0  ...      0.5        3.00           0.0
2             1  ...      0.5        3.14           0.0
3             0  ...      0.5        0.00           0.0
4             1  ...      0.5        0.00           0.0

   improvement_surcharge  total_amount  congestion_surcharge
Airport_fee  \
0                    1.0         33.96                   2.5
0.0
1                    1.0         29.43                   NaN
NaN
2                    1.0         18.84                   2.5
0.0
3                    1.0         21.22                   NaN
NaN
4                    1.0         10.10                   2.5
0.0

         date  hour airport_fee
0  2023-12-01     0         NaN
```

```
1  2023-12-01     0          NaN
2  2023-12-01     0          NaN
3  2023-12-01     0          NaN
4  2023-12-01     0          NaN

[5 rows x 22 columns]
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1896400 entries, 0 to 1896399
Data columns (total 22 columns):
 #   Column                 Dtype
---  ------                 -----
 0   VendorID               int64
 1   tpep_pickup_datetime   datetime64[us]
 2   tpep_dropoff_datetime  datetime64[us]
 3   passenger_count        float64
 4   trip_distance          float64
 5   RatecodeID             float64
 6   store_and_fwd_flag     object
 7   PULocationID           int64
 8   DOLocationID           int64
 9   payment_type           int64
 10  fare_amount            float64
 11  extra                  float64
 12  mta_tax                float64
 13  tip_amount             float64
 14  tolls_amount           float64
 15  improvement_surcharge  float64
 16  total_amount           float64
 17  congestion_surcharge   float64
 18  Airport_fee            float64
 19  date                   object
 20  hour                   int32
 21  airport_fee            float64
dtypes: datetime64[us](2), float64(13), int32(1), int64(4), object(2)
memory usage: 311.1+ MB
```

2.1 Fixing Columns

[10 marks]

Fix/drop any columns as you seem necessary in the below sections

2.1.1 [2 marks]

Fix the index and drop unnecessary columns

```
# Fix the index and drop any columns that are not needed
#Fix the index
df = df.reset_index(drop=True)
#Drop any columns
#column store_and_fwd_flag can be dropped
df = df.drop(columns=['store_and_fwd_flag'], errors='ignore')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1896400 entries, 0 to 1896399
Data columns (total 21 columns):
 #   Column                 Dtype
---  ------                 -----
 0   VendorID               int64
 1   tpep_pickup_datetime   datetime64[us]
 2   tpep_dropoff_datetime  datetime64[us]
 3   passenger_count        float64
 4   trip_distance          float64
 5   RatecodeID             float64
 6   PULocationID           int64
 7   DOLocationID           int64
 8   payment_type           int64
 9   fare_amount            float64
 10  extra                  float64
 11  mta_tax                float64
 12  tip_amount             float64
 13  tolls_amount           float64
 14  improvement_surcharge  float64
 15  total_amount           float64
 16  congestion_surcharge   float64
 17  Airport_fee            float64
 18  date                   object
 19  hour                   int32
 20  airport_fee            float64
dtypes: datetime64[us](2), float64(13), int32(1), int64(4), object(1)
memory usage: 296.6+ MB
```

2.1.2 [3 marks] There are two airport fee columns. This is possibly an error in naming columns. Let's see whether these can be combined into a single column.

```
# Combine the two airport fee columns
df["Airport_fee"] = df["airport_fee"].fillna(0) +
df["Airport_fee"].fillna(0)  # Merge values into one single column
Airport_fee
df.drop(columns=["airport_fee"], inplace=True)  # Drop the extra
column
df.info()
#combines two similarly named columns, 'airport_fee' and
'Airport_fee', by summing their values into a single 'Airport_fee'
```

```
column, then safely removes the duplicate 'airport_fee' column to
clean the dataset

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1896400 entries, 0 to 1896399
Data columns (total 20 columns):
 #   Column                 Dtype
---  ------                 -----
 0   VendorID               int64
 1   tpep_pickup_datetime   datetime64[us]
 2   tpep_dropoff_datetime  datetime64[us]
 3   passenger_count        float64
 4   trip_distance          float64
 5   RatecodeID             float64
 6   PULocationID           int64
 7   DOLocationID           int64
 8   payment_type           int64
 9   fare_amount            float64
 10  extra                  float64
 11  mta_tax                float64
 12  tip_amount             float64
 13  tolls_amount           float64
 14  improvement_surcharge  float64
 15  total_amount           float64
 16  congestion_surcharge   float64
 17  Airport_fee            float64
 18  date                   object
 19  hour                   int32
dtypes: datetime64[us](2), float64(12), int32(1), int64(4), object(1)
memory usage: 282.1+ MB
```

2.1.3 [5 marks] Fix columns with negative (monetary) values

```
# check where values of fare amount are negative
df[df["fare_amount"] < 0.0] #No negative Fare Amount

Empty DataFrame
Columns: [VendorID, tpep_pickup_datetime, tpep_dropoff_datetime,
passenger_count, trip_distance, RatecodeID, PULocationID,
DOLocationID, payment_type, fare_amount, extra, mta_tax, tip_amount,
tolls_amount, improvement_surcharge, total_amount,
congestion_surcharge, Airport_fee, date, hour]
Index: []
```

Did you notice something different in the `RatecodeID` column for above records?

```
# Analyse RatecodeID for the negative fare amounts
df[df["fare_amount"] < 0.0]["RatecodeID"].value_counts()
```

```
Series([], Name: count, dtype: int64)
```

```
# Find which columns have negative values
negative_columns =
df.select_dtypes(include=['number']).columns[(df.select_dtypes(include
=['number']) < 0).any()]
print(negative_columns)
```

```
Index(['extra', 'mta_tax', 'improvement_surcharge', 'total_amount',
       'congestion_surcharge', 'Airport_fee'],
      dtype='object')
```

```
# fix these negative values
#Column Extra
df[df["extra"] < 0]
```

```
        VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
683060         2  2023-11-06 22:37:04   2023-11-06 22:37:55
1.0
846135         2  2023-10-06 22:24:42   2023-10-06 22:25:38
2.0
961916         2  2023-10-27 14:51:03   2023-10-27 14:51:11
1.0

        trip_distance  RatecodeID  PULocationID  DOLocationID
payment_type  \
683060           0.03         1.0           229           229
2
846135           0.03         1.0           161           161
2
961916           0.00         1.0           265           265
2

        fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
683060          0.0   -1.0     -0.5         0.0           0.0
846135          0.0   -1.0     -0.5         0.0           0.0
961916          3.0   -2.5      0.0         0.0           0.0

        improvement_surcharge  total_amount  congestion_surcharge  \
683060                   -1.0          -5.0                  -2.5
846135                   -1.0          -5.0                  -2.5
961916                    1.0           4.0                   0.0

        Airport_fee        date  hour
683060          0.0  2023-11-06    22
846135          0.0  2023-10-06    22
961916          0.0  2023-10-27    14
```

```
# As there are only 3 rows which has negative values in any of the
columns, we can drop these 3 rows which will have no effect in our
```

```
dataset.
```

2.2 Handling Missing Values

[10 marks]

2.2.1 [2 marks] Find the proportion of missing values in each column

```
# Find the proportion of missing values in each column
df.isnull().mean()*100
```

```
VendorID                 0.000000
tpep_pickup_datetime     0.000000
tpep_dropoff_datetime    0.000000
passenger_count          0.000000
trip_distance            0.000000
RatecodeID               3.420903
PULocationID             0.000000
DOLocationID             0.000000
payment_type             0.000000
fare_amount              0.000000
extra                    0.000000
mta_tax                  0.000000
tip_amount               0.000000
tolls_amount             0.000000
improvement_surcharge    0.000000
total_amount             0.000000
congestion_surcharge     3.420903
Airport_fee              0.000000
date                     0.000000
hour                     0.000000
dtype: float64
```

2.2.2 [3 marks] Handling missing values in `passenger_count`

```
# Display the rows with null values
# Impute NaN values in 'passenger_count'
#Replacing NaN values with 1.0 because every ride must have at least
one passenger (a taxi cannot be empty).
df["passenger_count"].fillna(1.0, inplace=True)
df.isnull().sum()
```

```
VendorID                     0
tpep_pickup_datetime         0
tpep_dropoff_datetime        0
passenger_count              0
trip_distance                0
RatecodeID               64874
```

```
PULocationID                0
DOLocationID                0
payment_type                0
fare_amount                 0
extra                       0
mta_tax                     0
tip_amount                  0
tolls_amount                0
improvement_surcharge       0
total_amount                0
congestion_surcharge    64874
Airport_fee                 0
date                        0
hour                        0
dtype: int64
```

```
df[df["passenger_count"]==0]
```

```
Empty DataFrame
Columns: [VendorID, tpep_pickup_datetime, tpep_dropoff_datetime,
passenger_count, trip_distance, RatecodeID, PULocationID,
DOLocationID, payment_type, fare_amount, extra, mta_tax, tip_amount,
tolls_amount, improvement_surcharge, total_amount,
congestion_surcharge, Airport_fee, date, hour]
Index: []
```

```
 #There are 29681 records with zero values in passenger count ,as
mentioned earlier every ride must atleast have 1 passenger and the
most occurance of passenger count is also 1.0 hence replacing it with
1
df.loc[df['passenger_count'] == 0, 'passenger_count'] = 1.0
df[df["passenger_count"]==0]
```

```
Empty DataFrame
Columns: [VendorID, tpep_pickup_datetime, tpep_dropoff_datetime,
passenger_count, trip_distance, RatecodeID, PULocationID,
DOLocationID, payment_type, fare_amount, extra, mta_tax, tip_amount,
tolls_amount, improvement_surcharge, total_amount,
congestion_surcharge, Airport_fee, date, hour]
Index: []
```

Did you find zeroes in passenger_count? Handle these.

2.2.3 [2 marks] Handle missing values in `RatecodeID`

```
# Fix missing values in 'RatecodeID'
df.isnull().sum()
```

```
VendorID                    0
tpep_pickup_datetime        0
```

```
tpep_dropoff_datetime         0
passenger_count               0
trip_distance                 0
RatecodeID                64874
PULocationID                  0
DOLocationID                  0
payment_type                  0
fare_amount                   0
extra                         0
mta_tax                       0
tip_amount                    0
tolls_amount                  0
improvement_surcharge         0
total_amount                  0
congestion_surcharge      64874
Airport_fee                   0
date                          0
hour                          0
dtype: int64
```

```
df["RatecodeID"].value_counts()
```

```
RatecodeID
1.0     1729259
2.0       71670
99.0      10472
5.0       10275
3.0        6124
4.0        3723
6.0           3
Name: count, dtype: int64
```

```
#since RatecodeID is categorical (1-6 codes), replacing NaN with the
most common value (mode) is a good approach
df["RatecodeID"].fillna(df["RatecodeID"].mode()[0], inplace=True)
```

2.2.4 [3 marks] Impute NaN in `congestion_surcharge`

```
# handle null values in congestion_surcharge
df.isnull().sum()
```

```
VendorID                      0
tpep_pickup_datetime          0
tpep_dropoff_datetime         0
passenger_count               0
trip_distance                 0
RatecodeID                    0
PULocationID                  0
DOLocationID                  0
payment_type                  0
```

```
fare_amount                0
extra                      0
mta_tax                    0
tip_amount                 0
tolls_amount               0
improvement_surcharge      0
total_amount               0
congestion_surcharge     64874
Airport_fee                0
date                       0
hour                       0
dtype: int64
```

```
df["congestion_surcharge"].value_counts()
```

```
congestion_surcharge
 2.5    1690572
 0.0     140897
-2.5         56
 0.5          1
Name: count, dtype: int64
```

```
#Replacing NaN with the most common value (mode) with
Congestion_surcharge
df["congestion_surcharge"].fillna(df["congestion_surcharge"].mode()
[0], inplace=True)
```

Are there missing values in other columns? Did you find NaN values in some other set of columns? Handle those missing values below.

```
# Handle any remaining missing values

# Handle any remaining missing values
df.isnull().sum()
```

```
VendorID                 0
tpep_pickup_datetime     0
tpep_dropoff_datetime    0
passenger_count          0
trip_distance            0
RatecodeID               0
PULocationID             0
DOLocationID             0
payment_type             0
fare_amount              0
extra                    0
mta_tax                  0
tip_amount               0
tolls_amount             0
improvement_surcharge    0
```

```
total_amount              0
congestion_surcharge      0
Airport_fee               0
date                      0
hour                      0
dtype: int64
```

2.3 Handling Outliers

[10 marks]

Before we start fixing outliers, let's perform outlier analysis.

```
# Describe the data and check if there are any potential outliers
present
# Check for potential out of place values in various columns
df.describe()
```

```
           VendorID        tpep_pickup_datetime
tpep_dropoff_datetime  \
count  1.896400e+06                     1896400
1896400
mean   1.733026e+00  2023-07-02 19:59:52.930795  2023-07-02
20:17:18.919564
min    1.000000e+00         2022-12-31 23:51:30         2022-12-31
23:56:06
25%    1.000000e+00  2023-04-02 16:10:08.750000  2023-04-02
16:27:43.500000
50%    2.000000e+00  2023-06-27 15:44:22.500000         2023-06-27
16:01:15
75%    2.000000e+00         2023-10-06 19:37:45         2023-10-06
19:53:39
max    6.000000e+00         2023-12-31 23:57:51         2024-01-01
20:50:55
std    4.476401e-01                         NaN
NaN

       passenger_count  trip_distance    RatecodeID  PULocationID  \
count     1.896400e+06   1.896400e+06  1.896400e+06  1.896400e+06
mean      1.372236e+00   3.858293e+00  1.612981e+00  1.652814e+02
min       1.000000e+00   0.000000e+00  1.000000e+00  1.000000e+00
25%       1.000000e+00   1.050000e+00  1.000000e+00  1.320000e+02
50%       1.000000e+00   1.790000e+00  1.000000e+00  1.620000e+02
75%       1.000000e+00   3.400000e+00  1.000000e+00  2.340000e+02
max       9.000000e+00   1.263605e+05  9.900000e+01  2.650000e+02
std       8.644038e-01   1.294085e+02  7.267261e+00  6.400038e+01

       DOLocationID  payment_type   fare_amount         extra
mta_tax  \
count  1.896400e+06  1.896400e+06  1.896400e+06  1.896400e+06
```

```
1.896400e+06
mean    1.640515e+02  1.163817e+00  1.991935e+01  1.588018e+00
4.952796e-01
min     1.000000e+00  0.000000e+00  0.000000e+00 -2.500000e+00 -
5.000000e-01
25%     1.140000e+02  1.000000e+00  9.300000e+00  0.000000e+00
5.000000e-01
50%     1.620000e+02  1.000000e+00  1.350000e+01  1.000000e+00
5.000000e-01
75%     2.340000e+02  1.000000e+00  2.190000e+01  2.500000e+00
5.000000e-01
max     2.650000e+02  4.000000e+00  1.431635e+05  2.080000e+01
4.000000e+00
std     6.980207e+01  5.081384e-01  1.055371e+02  1.829200e+00
4.885128e-02

          tip_amount  tolls_amount  improvement_surcharge  total_amount
\
count  1.896400e+06  1.896400e+06           1.896400e+06  1.896400e+06

mean   3.547011e+00  5.965338e-01           9.989706e-01  2.898186e+01

min    0.000000e+00  0.000000e+00          -1.000000e+00 -5.750000e+00

25%    1.000000e+00  0.000000e+00           1.000000e+00  1.596000e+01

50%    2.850000e+00  0.000000e+00           1.000000e+00  2.100000e+01

75%    4.420000e+00  0.000000e+00           1.000000e+00  3.094000e+01

max    2.230800e+02  1.430000e+02           1.000000e+00  1.431675e+05

std    4.054882e+00  2.187878e+00           3.112072e-02  1.064162e+02


       congestion_surcharge   Airport_fee          hour
count          1.896400e+06  1.896400e+06  1.896400e+06
mean           2.314109e+00  1.380093e-01  1.426504e+01
min           -2.500000e+00 -1.750000e+00  0.000000e+00
25%            2.500000e+00  0.000000e+00  1.100000e+01
50%            2.500000e+00  0.000000e+00  1.500000e+01
75%            2.500000e+00  0.000000e+00  1.900000e+01
max            2.500000e+00  1.750000e+00  2.300000e+01
std            6.561568e-01  4.575896e-01  5.807381e+00
```

2.3.1 [10 marks] Based on the above analysis, it seems that some of the outliers are present due to errors in registering the trips. Fix the outliers.

Some points you can look for:

- Entries where `trip_distance` is nearly 0 and `fare_amount` is more than 300

- Entries where `trip_distance` and `fare_amount` are 0 but the pickup and dropoff zones are different (both distance and fare should not be zero for different zones)
- Entries where `trip_distance` is more than 250 miles.
- Entries where `payment_type` is 0 (there is no payment_type 0 defined in the data dictionary)

These are just some suggestions. You can handle outliers in any way you wish, using the insights from above outlier analysis.

How will you fix each of these values? Which ones will you drop and which ones will you replace?

First, let us remove 7+ passenger counts as there are very less instances.

```
# remove passenger_count > 6
df[df["passenger_count"] > 6]
```

```
         VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
20250           2  2023-12-04 15:32:03   2023-12-04 15:32:08
9.0
54713           2  2023-12-09 22:01:38   2023-12-09 22:01:40
8.0
121042          2  2023-12-20 19:26:27   2023-12-20 19:33:17
9.0
133422          2  2023-12-22 23:00:21   2023-12-22 23:00:24
8.0
139944          2  2023-12-26 17:38:04   2023-12-26 17:38:06
8.0
225079          2  2023-04-09 09:22:54   2023-04-09 09:23:22
7.0
293758          2  2023-04-21 16:44:17   2023-04-21 16:44:19
8.0
313674          2  2023-04-28 02:24:47   2023-04-28 02:25:01
8.0
489535          2  2023-05-29 02:35:04   2023-05-29 02:35:16
7.0
544416          2  2023-02-08 23:26:39   2023-02-08 23:26:51
9.0
606527          2  2023-02-19 17:19:13   2023-02-19 17:57:24
9.0
807103          2  2023-11-30 00:13:36   2023-11-30 00:13:39
8.0
891884          2  2023-10-15 08:11:40   2023-10-15 08:39:01
7.0
922435          2  2023-10-20 16:55:31   2023-10-20 16:56:27
7.0
1056785         2  2023-08-16 06:10:57   2023-08-16 06:49:47
8.0
1079217         2  2023-08-21 01:53:09   2023-08-21 01:53:11
8.0
```

```
1215971         2  2023-01-19 16:33:22   2023-01-19 17:57:32
8.0
1347424         2  2023-07-16 16:33:55   2023-07-16 16:34:00
8.0
1514385         2  2023-09-18 13:07:26   2023-09-18 14:05:27
8.0
1515631         2  2023-09-18 17:26:13   2023-09-18 17:52:25
7.0
1791519         2  2023-06-11 11:53:08   2023-06-11 11:53:29
9.0

         trip_distance  RatecodeID  PULocationID  DOLocationID
payment_type  \
20250             0.00         5.0           132           132
1
54713             0.00         5.0            79           264
1
121042            0.07         5.0           112           112
2
133422            0.09         5.0           236           236
1
139944            0.45         5.0           216           264
1
225079            0.00         5.0           125           125
1
293758            0.00         5.0           264           264
1
313674            0.00         5.0            87            87
2
489535            0.00         5.0           256           256
1
544416            0.13         5.0           231           231
1
606527           16.79         5.0           186             1
1
807103            0.00         5.0            90           264
1
891884            7.60         5.0           246           195
1
922435           16.17         5.0           132           132
1
1056785          19.03         5.0           132            75
1
1079217           0.00         5.0           264           264
2
1215971          18.30         5.0           230           132
1
1347424           0.00         5.0           233           233
1
```

```
1514385          31.71         5.0            48            219
1
1515631           5.11         5.0           246            265
1
1791519           0.00         5.0           138            138
1
```

```
         fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
20250          90.00    0.0      0.5       18.65          0.00
54713          87.00    0.0      0.5       17.70          0.00
121042         92.00    0.0      0.5        0.00          0.00
133422         85.00    0.0      0.5       17.30          0.00
139944         85.00    0.0      0.5       17.30          0.00
225079         80.00    0.0      0.5        0.00         21.25
293758         86.00    0.0      0.0       10.10          0.00
313674         85.00    0.0      0.5        0.00          0.00
489535         75.00    0.0      0.0        0.02          0.00
544416         95.55    0.0      0.5       19.91          0.00
606527         90.00    0.0      0.0       18.00         14.75
807103         86.00    0.0      0.5        5.00          0.00
891884         70.00    0.0      0.5       18.50          0.00
922435         74.00    0.0      0.0        0.00         13.00
1056785        82.00    0.0      0.5       15.00          6.94
1079217        82.00    0.0      0.0        0.00          0.00
1215971        85.00    0.0      0.5       19.11          6.55
1347424        88.00    0.0      0.5       17.90          0.00
1514385        88.90    0.0      0.5       10.00         11.19
1515631        70.00    0.0      0.0        0.00          0.00
1791519        95.00    5.0      0.0        0.00          0.00
```

```
         improvement_surcharge  total_amount  congestion_surcharge  \
20250                      1.0        111.90                   0.0
54713                      1.0        106.20                   0.0
121042                     1.0         93.50                   0.0
133422                     1.0        103.80                   0.0
139944                     1.0        103.80                   0.0
225079                     1.0        105.25                   2.5
293758                     1.0         97.10                   0.0
313674                     1.0         89.00                   2.5
489535                     1.0         76.02                   0.0
544416                     1.0        119.46                   2.5
606527                     1.0        123.75                   0.0
807103                     1.0         92.50                   0.0
891884                     1.0         92.50                   2.5
922435                     1.0         88.00                   0.0
1056785                    1.0        105.44                   0.0
1079217                    1.0         83.00                   0.0
1215971                    1.0        114.66                   2.5
1347424                    1.0        107.40                   0.0
```

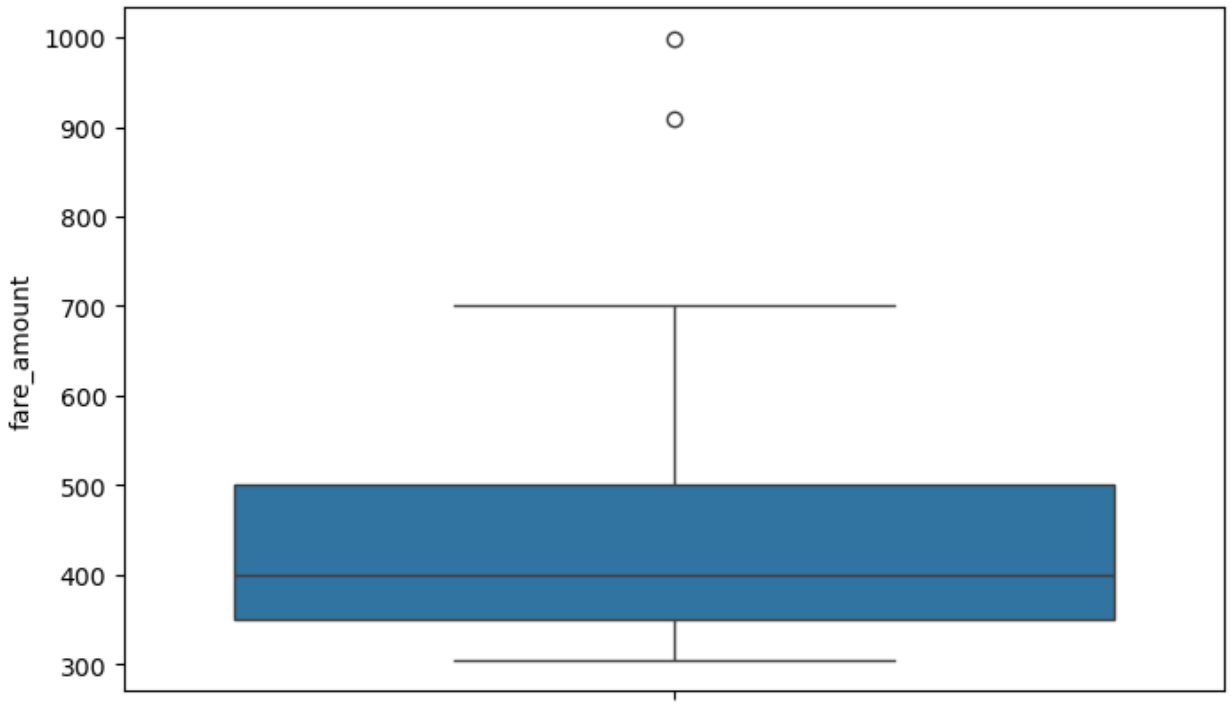
```
1514385                   1.0        114.09                   2.5
1515631                   1.0         71.00                   0.0
1791519                   1.0        102.75                   0.0
```

```
         Airport_fee        date  hour
20250           1.75  2023-12-04    15
54713           0.00  2023-12-09    22
121042          0.00  2023-12-20    19
133422          0.00  2023-12-22    23
139944          0.00  2023-12-26    17
225079          0.00  2023-04-09     9
293758          0.00  2023-04-21    16
313674          0.00  2023-04-28     2
489535          0.00  2023-05-29     2
544416          0.00  2023-02-08    23
606527          0.00  2023-02-19    17
807103          0.00  2023-11-30     0
891884          0.00  2023-10-15     8
922435          0.00  2023-10-20    16
1056785         0.00  2023-08-16     6
1079217         0.00  2023-08-21     1
1215971         0.00  2023-01-19    16
1347424         0.00  2023-07-16    16
1514385         0.00  2023-09-18    13
1515631         0.00  2023-09-18    17
1791519         1.75  2023-06-11    11
```

```
df.shape[0]#1896400
df=df[~(df["passenger_count"] > 6)] #21
df.shape[0]#1896379
```

```
1896379
```

```
# Continue with outlier handling
#Entries where trip_distance is nearly 0 and fare_amount is more than
300
outliers1 = df[(df["trip_distance"] <= 0.01) & (df["fare_amount"] >
300)]

# Create a boxplot for the fare_amount column
plt.figure(figsize=(8, 5))
sns.boxplot(y=outliers1["fare_amount"])
plt.show()
```




```
#There are outliers above $700, values reaching  $900, and even $1000
for trip_distance <=0.01.
df[(df["trip_distance"] <= 0.01) & (df["fare_amount"] > 700)]
```

```
         VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
544873          1  2023-02-09 07:37:30   2023-02-09 07:39:13
1.0
1165259         1  2023-01-09 16:17:32   2023-01-09 16:20:41
1.0

         trip_distance  RatecodeID  PULocationID  DOLocationID
payment_type  \
544873             0.0         5.0           246           246
4
1165259            0.0         5.0           141           141
3

         fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
544873         910.0    0.0      0.0         0.0           0.0
1165259        999.0    0.0      0.0         0.0           0.0

         improvement_surcharge  total_amount  congestion_surcharge  \
544873                     1.0         911.0                   0.0
1165259                    1.0        1000.0                   0.0

         Airport_fee        date  hour
```

```
544873           0.0  2023-02-09     7
1165259          0.0  2023-01-09    16

#High fare amount for 0.00  distance and pickup and drop time is also
minimal ,so its better to remove these records from the sample
df.shape[0]#1896379
df=df[~((df["trip_distance"] <= 0.01) & (df["fare_amount"] > 700))]#4
df.shape[0]#1896377

1896377

#Entries where trip_distance and fare_amount are 0 but the pickup and
dropoff zones are different (both distance and fare should not be zero
for different zones)
df[(df["trip_distance"] == 0) & (df["fare_amount"] == 0) &
(df["PULocationID"] != df["DOLocationID"])]

         VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
17263           1  2023-12-03 21:22:01   2023-12-03 21:22:01
1.0
49571           2  2023-12-09 08:34:26   2023-12-09 08:34:35
1.0
92393           1  2023-12-15 21:26:17   2023-12-15 21:26:17
1.0
106129          1  2023-12-18 09:31:12   2023-12-18 09:31:12
1.0
118468          1  2023-12-20 13:40:27   2023-12-20 13:40:27
1.0
...           ...                  ...                   ...
...
1776031         1  2023-06-08 16:33:15   2023-06-08 16:33:15
1.0
1806767         1  2023-06-14 09:40:39   2023-06-14 09:40:39
1.0
1815922         1  2023-06-15 17:56:56   2023-06-15 18:20:13
1.0
1863919         1  2023-06-24 16:05:27   2023-06-24 16:05:27
1.0
1885762         1  2023-06-28 21:29:09   2023-06-28 21:32:08
1.0

         trip_distance  RatecodeID  PULocationID  DOLocationID
payment_type  \
17263              0.0         5.0           186           264
2
49571              0.0         2.0           107           137
2
92393              0.0         5.0           239           264
2
```

```
106129              0.0         5.0           138            264
3
118468              0.0         5.0           138            264
2
...                 ...         ...           ...            ...
...
1776031             0.0         5.0           132            264
2
1806767             0.0         5.0           265            264
2
1815922             0.0         1.0            83             28
1
1863919             0.0         5.0           166            264
2
1885762             0.0         1.0           246             90
1

         fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
17263            0.0   0.00      0.0         0.0           0.0
49571            0.0   0.00     -0.5         0.0           0.0
92393            0.0   0.00      0.0         0.0           0.0
106129           0.0   0.00      0.0         0.0           0.0
118468           0.0   9.25      0.0         0.0           0.0
...              ...    ...      ...         ...           ...
1776031          0.0   0.00      0.0         0.0           0.0
1806767          0.0   0.00      0.0         0.0           0.0
1815922          0.0   0.00      0.0         0.0           0.0
1863919          0.0   0.00      0.0         0.0           0.0
1885762          0.0   0.00      0.0         0.0           0.0

         improvement_surcharge  total_amount  congestion_surcharge  \
17263                      0.0          0.00                   0.0
49571                     -1.0         -4.00                  -2.5
92393                      0.0          0.00                   0.0
106129                     0.0          0.00                   0.0
118468                     1.0         10.25                   2.5
...                        ...           ...                   ...
1776031                    1.0          1.00                   0.0
1806767                    1.0          1.00                   0.0
1815922                    0.0          0.00                   0.0
1863919                    0.0          0.00                   0.0
1885762                    0.0          0.00                   0.0

         Airport_fee        date  hour
17263           0.00  2023-12-03    21
49571           0.00  2023-12-09     8
92393           0.00  2023-12-15    21
106129          0.00  2023-12-18     9
118468          1.75  2023-12-20    13
...              ...         ...   ...
```

```
1776031         0.00  2023-06-08    16
1806767         0.00  2023-06-14     9
1815922         0.00  2023-06-15    17
1863919         0.00  2023-06-24    16
1885762         0.00  2023-06-28    21

[63 rows x 20 columns]
```

```
#it indicates data inconsistency,both trip distance and fare amount is
0.Fare amount can be adjusted by total fare-(all other taxes and
fares),but we cannot estimate trip distance as its only 63 records we
can delete it.
df.shape[0]#1896377
df=df[~((df["trip_distance"] <= 0.0) & (df["fare_amount"] == 0) &
(df["PULocationID"] != df["DOLocationID"]))]#63 records
df.shape[0]#1896314

#Entries where trip_distance is more than 250 miles.
df[df["trip_distance"] > 250.0]
```

```
         VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
48244           2  2023-12-08 23:45:00   2023-12-09 00:14:00
1.0
55714           2  2023-12-10 01:11:00   2023-12-10 01:25:00
1.0
59179           2  2023-12-10 17:10:00   2023-12-10 17:12:00
1.0
141227          1  2023-12-27 06:00:00   2023-12-27 07:22:13
1.0
160362          2  2023-12-30 13:24:39   2023-12-30 14:07:52
1.0
229852          2  2023-04-17 10:56:00   2023-04-17 11:25:00
1.0
299004          2  2023-04-22 14:58:35   2023-04-22 15:23:03
1.0
337101          2  2023-05-08 15:22:51   2023-05-08 16:02:16
1.0
374007          2  2023-05-11 19:43:07   2023-05-11 20:01:07
1.0
400900          2  2023-05-12 15:12:50   2023-05-12 16:03:58
1.0
471145          2  2023-05-25 11:10:00   2023-05-25 11:35:00
1.0
478458          2  2023-05-26 16:22:00   2023-05-26 16:56:00
1.0
490627          2  2023-05-29 13:13:00   2023-05-29 14:23:00
1.0
532931          2  2023-02-06 19:56:52   2023-02-06 20:33:55
2.0
```

```
554321          2  2023-02-10 19:53:45   2023-02-10 20:01:48
2.0
578301          2  2023-02-15 13:06:00   2023-02-15 13:40:00
1.0
587796          2  2023-02-17 07:17:00   2023-02-17 07:25:00
1.0
592588          2  2023-02-17 22:36:00   2023-02-17 23:00:00
1.0
607737          2  2023-02-19 22:06:00   2023-02-19 22:22:00
1.0
761434          2  2023-11-20 11:46:00   2023-11-20 12:30:00
1.0
813380          2  2023-10-01 00:05:00   2023-10-01 00:19:00
1.0
978126          2  2023-10-30 07:13:00   2023-10-30 07:33:00
1.0
1068338         1  2023-08-18 14:26:43   2023-08-18 15:04:09
1.0
1157612         2  2023-01-07 20:02:05   2023-01-07 20:07:10
1.0
1238616         2  2023-01-24 06:27:00   2023-01-24 07:18:00
1.0
1264000         2  2023-01-28 18:16:37   2023-01-28 18:41:22
1.0
1264763         2  2023-01-28 20:39:00   2023-01-28 20:59:00
1.0
1317299         2  2023-07-10 17:33:19   2023-07-10 19:14:56
1.0
1328763         2  2023-07-12 21:05:00   2023-07-12 21:10:00
1.0
1332219         2  2023-07-13 15:38:02   2023-07-13 16:09:33
1.0
1337354         2  2023-07-14 15:32:30   2023-07-14 16:18:58
1.0
1364484         2  2023-07-20 04:22:00   2023-07-20 04:42:00
1.0
1405216         2  2023-07-28 08:10:00   2023-07-28 08:47:00
1.0
1470580         2  2023-09-10 13:44:00   2023-09-10 14:16:00
1.0
1508910         1  2023-09-17 07:23:50   2023-09-17 07:48:20
1.0
1521242         2  2023-09-19 19:36:56   2023-09-19 19:48:16
1.0
1539683         2  2023-09-26 19:41:43   2023-09-26 20:01:00
1.0
1577960         2  2023-03-02 15:45:34   2023-03-02 16:00:45
1.0
1591156         2  2023-03-03 19:47:00   2023-03-03 20:05:00
```

```
1.0
1618417         2  2023-03-10 19:12:22   2023-03-10 19:18:55
1.0
1723560         2  2023-03-30 14:07:00   2023-03-30 15:32:00
1.0
1745502         2  2023-06-03 06:23:00   2023-06-03 06:31:00
1.0
1800833         2  2023-06-13 09:59:00   2023-06-13 10:12:00
1.0
1849443         2  2023-06-22 06:34:00   2023-06-22 06:47:00
1.0
1872930         2  2023-06-26 13:45:44   2023-06-26 13:51:12
2.0
1896239         2  2023-06-30 23:40:00   2023-07-01 00:11:00
1.0

         trip_distance  RatecodeID  PULocationID  DOLocationID
payment_type  \
48244          22414.00         1.0            65           188
0
55714          35482.69         1.0           224            33
0
59179          33133.96         1.0           142           142
0
141227           969.10        99.0           258           265
1
160362          9021.10         1.0           132            49
1
229852          7914.69         1.0            89           227
0
299004           403.66         1.0           144           237
1
337101          9678.97         1.0           138           161
1
374007          9675.03         1.0            68           231
1
400900          9678.78         1.0           138           116
2
471145         10433.95         1.0           116           238
0
478458         27586.37         1.0           230            17
0
490627        126360.46         1.0           265           132
0
532931          3317.68         2.0           132           186
1
554321          9673.76         1.0           264           142
1
578301          2942.93         1.0           231           166
```

```
0
587796          8645.77          1.0             238             230
0
592588         11551.95          1.0              41             170
0
607737          6284.45          1.0             186             236
0
761434         22869.37          1.0             179             237
0
813380         34804.51          1.0             263             151
0
978126          4547.48          1.0             143             136
0
1068338        56823.80         99.0              71              39
1
1157612          721.26          1.0             145               7
1
1238616         3253.99          1.0             230              90
0
1264000         2003.03          1.0              48             113
0
1264763        10451.89          1.0             142             224
0
1317299         9717.11          1.0             216             265
4
1328763        30430.04          1.0             263             263
0
1332219         9679.36          1.0             138             234
1
1337354         9677.04          1.0             138             263
1
1364484        32053.26          1.0               4             138
0
1405216        22576.01          1.0             151             137
0
1470580        22910.92          1.0             163             223
0
1508910        10452.60         99.0             159             254
1
1521242         9674.07          1.0             233             186
1
1539683         9678.34          1.0             138             197
1
1577960         9674.01          1.0             161              68
1
1591156         7094.16          1.0              75              42
0
1618417        10961.43          1.0             140             263
2
```

```
1723560       12133.27         1.0            50            132
0
1745502       26217.98         1.0           231             68
0
1800833       22528.82         1.0           116            239
0
1849443       22562.67         1.0           151            237
0
1872930        6262.99         1.0            75            238
1
1896239       20314.00         1.0           237            248
0

         fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
48244          21.69    0.0      0.5        4.64          0.00
55714          25.91    0.0      0.5        4.49          0.00
59179          12.02    0.0      0.5        2.40          0.00
141227         25.50    0.0      0.5        0.00          0.00
160362         80.00    0.0      0.5        3.00          0.00
229852         19.85    0.0      0.5        0.00          0.00
299004         24.00    0.0      0.5        2.00          0.00
337101         45.00    0.0      0.5        2.39          6.55
374007         19.80    2.5      0.5        7.89          0.00
400900         47.10    0.0      0.5        0.00          6.55
471145         21.25    0.0      0.5        0.00          0.00
478458         46.43    0.0      0.5        0.00          6.55
490627         85.52    0.0      0.5       20.71          6.55
532931         70.00    5.0      0.5       17.11          6.55
554321          8.60    2.5      0.5        1.50          0.00
578301         26.49    0.0      0.5        4.63          0.00
587796         13.34    0.0      0.5        4.34          0.00
592588         24.03    0.0      0.5        5.61          0.00
607737         16.00    0.0      0.5        0.00          0.00
761434         27.39    0.0      0.5        0.00          0.00
813380         14.45    0.0      0.5        3.69          0.00
978126         36.47    0.0      0.5        4.05          0.00
1068338        18.50    0.0      0.5        0.00          0.00
1157612         7.90    0.0      0.5        2.00          0.00
1238616        19.24    0.0      0.5        4.51          0.00
1264000        18.87    0.0      0.5        3.00          0.00
1264763        19.94    0.0      0.5        4.79          0.00
1317299       264.80    2.5      0.5        0.00          0.00
1328763        23.01    0.0      0.5        0.00          0.00
1332219        40.10    5.0      0.5       10.00          6.55
1337354        52.70    5.0      0.5       13.15          6.55
1364484        36.90    0.0      0.5        8.18          0.00
1405216        28.19    0.0      0.5        3.00          0.00
1470580        27.66    0.0      0.5        6.33          0.00
1508910        27.50    0.0      0.5        0.00          0.00
```

1521242	11.40	2.5	0.5	4.47	0.00
1539683	29.60	7.5	0.5	7.72	0.00
1577960	14.20	0.0	0.5	2.00	0.00
1591156	18.81	0.0	0.5	0.00	0.00
1618417	8.60	2.5	0.5	0.00	0.00
1723560	77.00	0.0	0.5	16.20	0.00
1745502	10.63	0.0	0.5	2.93	0.00
1800833	17.42	0.0	0.5	0.37	0.00
1849443	16.32	0.0	0.5	2.03	0.00
1872930	7.90	0.0	0.5	1.88	0.00
1896239	31.47	0.0	0.5	7.09	0.00

	improvement_surcharge	total_amount	congestion_surcharge	\
48244	1.0	27.83	2.5	
55714	1.0	34.40	2.5	
59179	1.0	18.42	2.5	
141227	1.0	27.00	0.0	
160362	1.0	88.75	2.5	
229852	1.0	21.35	2.5	
299004	1.0	30.00	2.5	
337101	1.0	57.94	2.5	
374007	1.0	34.19	2.5	
400900	1.0	55.15	0.0	
471145	1.0	25.25	2.5	
478458	1.0	56.98	2.5	
490627	1.0	114.28	2.5	
532931	1.0	103.91	2.5	
554321	1.0	16.60	2.5	
578301	0.3	34.42	2.5	
587796	1.0	21.68	2.5	
592588	1.0	33.64	2.5	
607737	1.0	20.00	2.5	
761434	1.0	31.39	2.5	
813380	1.0	22.14	2.5	
978126	1.0	44.52	2.5	
1068338	1.0	20.00	0.0	
1157612	1.0	11.40	0.0	
1238616	0.3	27.05	2.5	
1264000	1.0	25.87	2.5	
1264763	1.0	28.73	2.5	
1317299	1.0	271.30	2.5	
1328763	1.0	27.01	2.5	
1332219	1.0	67.40	2.5	
1337354	1.0	80.65	0.0	
1364484	1.0	49.08	2.5	
1405216	1.0	35.19	2.5	
1470580	1.0	37.99	2.5	
1508910	1.0	29.00	0.0	
1521242	1.0	22.37	2.5	

1539683		1.0	48.07	0.0
1577960		1.0	20.20	2.5
1591156		1.0	20.31	2.5
1618417		1.0	15.10	2.5
1723560		1.0	97.20	2.5
1745502		1.0	17.56	2.5
1800833		1.0	21.79	2.5
1849443		1.0	22.35	2.5
1872930		1.0	11.28	0.0
1896239		1.0	42.56	2.5

	Airport_fee	date	hour
48244	0.00	2023-12-08	23
55714	0.00	2023-12-10	1
59179	0.00	2023-12-10	17
141227	0.00	2023-12-27	6
160362	1.75	2023-12-30	13
229852	0.00	2023-04-17	10
299004	0.00	2023-04-22	14
337101	0.00	2023-05-08	15
374007	0.00	2023-05-11	19
400900	0.00	2023-05-12	15
471145	0.00	2023-05-25	11
478458	0.00	2023-05-26	16
490627	0.00	2023-05-29	13
532931	1.25	2023-02-06	19
554321	0.00	2023-02-10	19
578301	0.00	2023-02-15	13
587796	0.00	2023-02-17	7
592588	0.00	2023-02-17	22
607737	0.00	2023-02-19	22
761434	0.00	2023-11-20	11
813380	0.00	2023-10-01	0
978126	0.00	2023-10-30	7
1068338	0.00	2023-08-18	14
1157612	0.00	2023-01-07	20
1238616	0.00	2023-01-24	6
1264000	0.00	2023-01-28	18
1264763	0.00	2023-01-28	20
1317299	0.00	2023-07-10	17
1328763	0.00	2023-07-12	21
1332219	1.75	2023-07-13	15
1337354	1.75	2023-07-14	15
1364484	0.00	2023-07-20	4
1405216	0.00	2023-07-28	8
1470580	0.00	2023-09-10	13
1508910	0.00	2023-09-17	7
1521242	0.00	2023-09-19	19
1539683	1.75	2023-09-26	19

```
1577960          0.00  2023-03-02    15
1591156          0.00  2023-03-03    19
1618417          0.00  2023-03-10    19
1723560          0.00  2023-03-30    14
1745502          0.00  2023-06-03     6
1800833          0.00  2023-06-13     9
1849443          0.00  2023-06-22     6
1872930          0.00  2023-06-26    13
1896239          0.00  2023-06-30    23
```

```
df.shape[0]#1896314
df=df[~(df["trip_distance"] > 250.0)]#46
df.shape[0]#1896268
```

```
1896268
```

```
#Entries where payment_type is 0 (there is no payment_type 0 defined
in the data dictionary)
df[df["payment_type"]==0]
```

```
         VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
1                2  2023-12-01 00:38:48   2023-12-01 01:01:55
1.0
3                2  2023-12-01 00:00:50   2023-12-01 00:14:37
1.0
27               2  2023-12-01 00:01:11   2023-12-01 00:15:53
1.0
122              2  2023-12-01 00:02:18   2023-12-01 00:12:25
1.0
127              1  2023-12-01 00:04:14   2023-12-01 00:25:16
1.0
...            ...                  ...                   ...
...
1896293          1  2023-06-30 23:14:07   2023-06-30 23:25:45
1.0
1896309          2  2023-06-30 23:40:46   2023-07-01 00:04:37
1.0
1896352          2  2023-06-30 23:57:33   2023-07-01 00:09:15
1.0
1896373          2  2023-06-30 23:36:40   2023-06-30 23:53:20
1.0
1896383          1  2023-06-30 23:34:22   2023-07-01 00:32:59
1.0

         trip_distance  RatecodeID  PULocationID  DOLocationID
payment_type  \
1                 4.79         1.0           231            61
0
3                 2.08         1.0           137           144
```

```
0
27                3.49         1.0           164            262
0
122               1.79         1.0           142            239
0
127               0.00         1.0           186             74
0
...                ...         ...           ...            ...
...
1896293           0.70         1.0           230            186
0
1896309           4.46         1.0           143             79
0
1896352           2.75         1.0           166            142
0
1896373           5.18         1.0           148            237
0
1896383          20.20         1.0           132             74
0

         fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
1              22.43   0.00      0.5        3.00          0.00
3              17.22   0.00      0.5        0.00          0.00
27             17.83   0.00      0.5        0.00          0.00
122             9.88   0.00      0.5        0.00          0.00
127            30.31   0.00      0.5        0.00          0.00
...              ...    ...      ...         ...           ...
1896293        11.40   1.00      0.5        2.46          0.00
1896309        23.26   0.00      0.5        0.00          0.00
1896352        16.14   0.00      0.5        0.00          0.00
1896373        26.09   0.00      0.5        3.01          0.00
1896383        70.00   1.75      0.5       11.97          6.55

         improvement_surcharge  total_amount  congestion_surcharge  \
1                          1.0         29.43                   2.5
3                          1.0         21.22                   2.5
27                         1.0         21.83                   2.5
122                        1.0         13.88                   2.5
127                        1.0         34.31                   2.5
...                        ...           ...                   ...
1896293                    1.0         18.86                   2.5
1896309                    1.0         27.26                   2.5
1896352                    1.0         20.14                   2.5
1896373                    1.0         33.10                   2.5
1896383                    1.0         91.77                   2.5

         Airport_fee        date  hour
1                0.0  2023-12-01     0
3                0.0  2023-12-01     0
27               0.0  2023-12-01     0
```

```
122              0.0  2023-12-01     0
127              0.0  2023-12-01     0
...              ...         ...   ...
1896293          0.0  2023-06-30    23
1896309          0.0  2023-06-30    23
1896352          0.0  2023-06-30    23
1896373          0.0  2023-06-30    23
1896383          0.0  2023-06-30    23

[64844 rows x 20 columns]
```

```
#There are 64844 records with payment_type as zero,that is almost
3.41% so deleting these records is not a wise decision ,instead of
this we can replace the value to 5 as its known as unknown
df.loc[df["payment_type"]==0, "payment_type"] = 5
df[df["payment_type"]==0] # 0 columns
```

```
Empty DataFrame
Columns: [VendorID, tpep_pickup_datetime, tpep_dropoff_datetime,
passenger_count, trip_distance, RatecodeID, PULocationID,
DOLocationID, payment_type, fare_amount, extra, mta_tax, tip_amount,
tolls_amount, improvement_surcharge, total_amount,
congestion_surcharge, Airport_fee, date, hour]
Index: []
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1896268 entries, 0 to 1896399
Data columns (total 20 columns):
 #   Column                 Dtype
---  ------                 -----
 0   VendorID               int64
 1   tpep_pickup_datetime   datetime64[us]
 2   tpep_dropoff_datetime  datetime64[us]
 3   passenger_count        float64
 4   trip_distance          float64
 5   RatecodeID             float64
 6   PULocationID           int64
 7   DOLocationID           int64
 8   payment_type           int64
 9   fare_amount            float64
 10  extra                  float64
 11  mta_tax                float64
 12  tip_amount             float64
 13  tolls_amount           float64
 14  improvement_surcharge  float64
 15  total_amount           float64
 16  congestion_surcharge   float64
 17  Airport_fee            float64
```

```
 18  date                  object
 19  hour                  int32
dtypes: datetime64[us](2), float64(12), int32(1), int64(4), object(1)
memory usage: 296.6+ MB

#Do any columns need standardising?
#Passenger_count can be changed from float to int as the content is a
whole number it can be changed into int
df["passenger_count"] = df["passenger_count"].astype(int)
```

3 Exploratory Data Analysis

[90 marks]

```
df.columns.tolist()

['VendorID',
 'tpep_pickup_datetime',
 'tpep_dropoff_datetime',
 'passenger_count',
 'trip_distance',
 'RatecodeID',
 'PULocationID',
 'DOLocationID',
 'payment_type',
 'fare_amount',
 'extra',
 'mta_tax',
 'tip_amount',
 'tolls_amount',
 'improvement_surcharge',
 'total_amount',
 'congestion_surcharge',
 'Airport_fee',
 'date',
 'hour']
```

3.1 General EDA: Finding Patterns and Trends

[40 marks]

3.1.1 [3 marks] Categorise the varaibles into Numerical or Categorical.

- `VendorID`:
- `tpep_pickup_datetime`:
- `tpep_dropoff_datetime`:
- `passenger_count`:
- `trip_distance`:
- `RatecodeID`:

- `PULocationID`:
- `DOLocationID`:
- `payment_type`:
- `pickup_hour`:
- `trip_duration`:

The following monetary parameters belong in the same category, is it categorical or numerical?

- `fare_amount`
- `extra`
- `mta_tax`
- `tip_amount`
- `tolls_amount`
- `improvement_surcharge`
- `total_amount`
- `congestion_surcharge`
- `airport_fee`

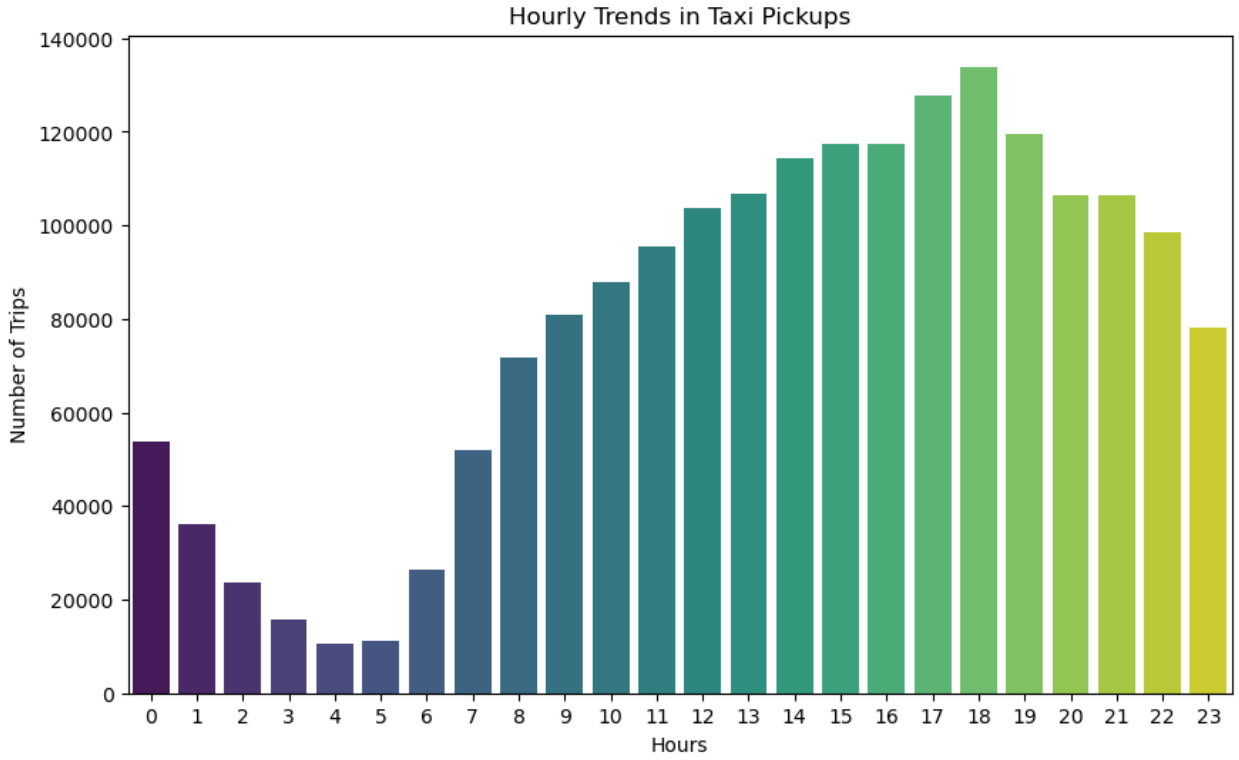
Temporal Analysis

3.1.2 [5 marks] Analyse the distribution of taxi pickups by hours, days of the week, and months.

```
# Find and show the hourly trends in taxi pickups
#we have dervied hour column from tpep_pickup_dateeime,we can group
the count based on hour
hourly_trends=df.groupby("hour").size()
print(hourly_trends)
plt.figure(figsize=(10,6))
sns.barplot(x=hourly_trends.index, y=hourly_trends.values,
palette="viridis")
plt.xlabel("Hours")
plt.ylabel("Number of Trips")
plt.title("Hourly Trends in Taxi Pickups")
plt.show()
```

```
hour
0      53675
1      36027
2      23766
3      15675
4      10634
5      11135
6      26322
7      51822
8      71809
9      80986
10     87909
11     95426
12    103585
```

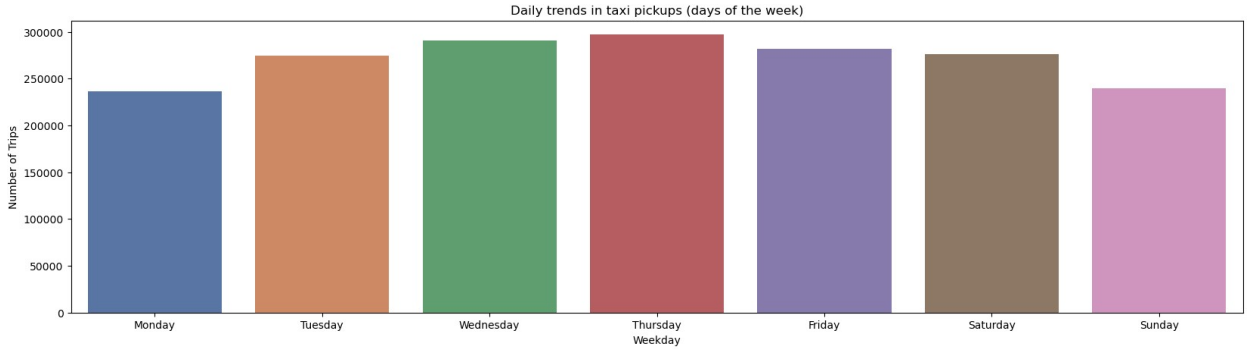
```
13    106799
14    114453
15    117397
16    117501
17    127859
18    133934
19    119652
20    106582
21    106494
22     98682
23     78144
dtype: int64
```




```
# Find and show the daily trends in taxi pickups (days of the week)
df["weekday"] = df["tpep_pickup_datetime"].dt.day_name()#to get the
day of week
weekdays_order = ["Monday", "Tuesday", "Wednesday", "Thursday",
"Friday", "Saturday", "Sunday"]
daily_trends=df.groupby("weekday").size().reindex(weekdays_order)
print(daily_trends)
plt.figure(figsize=(20,5))
sns.barplot(x=daily_trends.index, y=daily_trends.values,
palette="deep")
plt.xlabel("Weekday")
plt.ylabel("Number of Trips")
```

```
plt.title("Daily trends in taxi pickups (days of the week)")
plt.show()
```

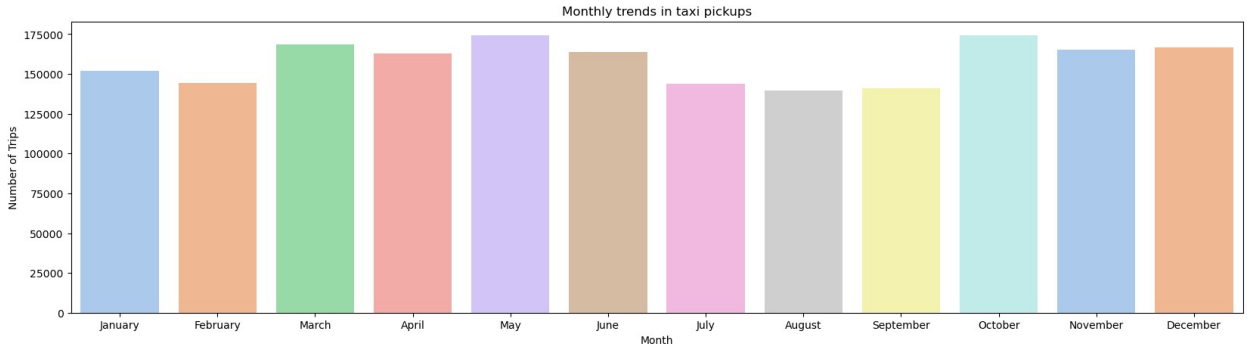
```
weekday
Monday        236300
Tuesday       274267
Wednesday     290493
Thursday      297287
Friday        282172
Saturday      276138
Sunday        239611
dtype: int64
```




```
# Show the monthly trends in pickups
df["Month"] = df["tpep_pickup_datetime"].dt.month_name()#to get the
month
month_order = ["January", "February", "March", "April", "May", "June",
"July","August","September","October","November","December"]
month_trends=df.groupby("Month").size().reindex(month_order)
print(month_trends)
plt.figure(figsize=(20,5))
sns.barplot(x=month_trends.index, y=month_trends.values,
palette="pastel")
plt.xlabel("Month")
plt.ylabel("Number of Trips")
plt.title("Monthly trends in taxi pickups")
plt.show()
```

```
Month
January       152075
February      144445
March         168689
April         162899
May           174057
June          163772
July          143771
August        139637
September     140867
```

```
October      174240
November     165124
December     166692
dtype: int64
```



Financial Analysis

Take a look at the financial parameters like `fare_amount`, `tip_amount`, `total_amount`, and also `trip_distance`. Do these contain zero/negative values?

```
# Analyse the above parameters
# Analyse the above parameters
df[df["fare_amount"]<=0]
df[df["tip_amount"]<=0]
df[df["total_amount"]<=0]
df[df["trip_distance"]<=0]
```

```
         VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
7                2  2023-12-01 00:36:28   2023-12-01 00:36:34
1
127              1  2023-12-01 00:04:14   2023-12-01 00:25:16
1
219              2  2023-12-01 01:08:22   2023-12-01 01:08:56
1
227              1  2023-12-01 01:03:11   2023-12-01 01:03:15
1
246              1  2023-12-01 01:08:05   2023-12-01 01:14:02
1
...            ...                  ...                   ...
...
1895952          2  2023-06-30 22:36:31   2023-06-30 22:36:34
2
1896028          1  2023-06-30 22:19:25   2023-06-30 22:29:51
1
1896172          1  2023-06-30 22:29:55   2023-06-30 22:30:20
1
1896277          2  2023-06-30 23:03:15   2023-06-30 23:10:06
```

```
5
1896396         1  2023-06-30 23:22:42   2023-06-30 23:39:06
1

         trip_distance  RatecodeID  PULocationID  DOLocationID
payment_type  \
7                  0.0         5.0           170           170
1
127                0.0         1.0           186            74
5
219                0.0         1.0           161           161
4
227                0.0         1.0           161           161
2
246                0.0         1.0           140           262
1
...                ...         ...           ...           ...
...
1895952            0.0         5.0           228           228
1
1896028            0.0         1.0           209             4
5
1896172            0.0         5.0           170           170
1
1896277            0.0         1.0           140           262
1
1896396            0.0        99.0            90           232
1

         fare_amount  ...  tip_amount  tolls_amount
improvement_surcharge  \
7              11.00  ...        2.90           0.0
1.0
127            30.31  ...        0.00           0.0
1.0
219             3.00  ...        0.00           0.0
1.0
227             3.00  ...        0.00           0.0
1.0
246             4.40  ...        2.00           0.0
1.0
...              ...  ...         ...           ...
...
1895952        45.00  ...        0.01           0.0
1.0
1896028        12.65  ...        0.00           0.0
1.0
1896172        19.00  ...        4.00           0.0
1.0
```

```
1896277          7.20  ...         1.22           0.0
1.0
1896396         18.20  ...         0.00           0.0
1.0

         total_amount  congestion_surcharge  Airport_fee        date
hour  \
7               17.40                   2.5          0.0  2023-12-01
0
127             34.31                   2.5          0.0  2023-12-01
0
219              8.00                   2.5          0.0  2023-12-01
1
227              8.00                   2.5          0.0  2023-12-01
1
246             11.40                   2.5          0.0  2023-12-01
1
...               ...                   ...          ...         ...
...
1895952         46.51                   0.0          0.0  2023-06-30
22
1896028         16.65                   2.5          0.0  2023-06-30
22
1896172         24.00                   0.0          0.0  2023-06-30
22
1896277         13.42                   2.5          0.0  2023-06-30
23
1896396         19.70                   0.0          0.0  2023-06-30
23

        weekday     Month
7        Friday  December
127      Friday  December
219      Friday  December
227      Friday  December
246      Friday  December
...         ...       ...
1895952  Friday      June
1896028  Friday      June
1896172  Friday      June
1896277  Friday      June
1896396  Friday      June

[37657 rows x 22 columns]
```

Do you think it is beneficial to create a copy DataFrame leaving out the zero values from these?

3.1.3 [2 marks] Filter out the zero values from the above columns.

Note: The distance might be 0 in cases where pickup and drop is in the same zone. Do you think it is suitable to drop such cases of zero distance?

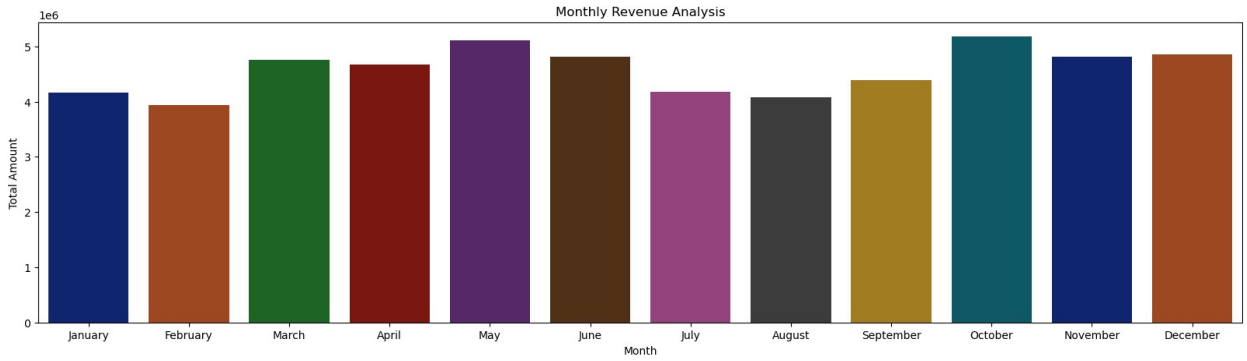
```
# Create a df with non zero entries for the selected parameters.
#Fare Amount
df[df["fare_amount"]<=0]#588 rows
#fare amount is a parameter which is calculated from the trip distance
and ratecodeid ,as we dont know the ratecode against the ratecodeid
value we can't fix these ones and as the number count is
588/1896268~0.03% we can delete it

df.shape[0]#1896268
df=df[~(df["fare_amount"] <=0)] #588
df.shape[0]#1873933
```

```
1895680
```

3.1.4 [3 marks] Analyse the monthly revenue (`total_amount`) trend

```
# Group data by month and analyse monthly revenue
monthly_revenue = df.groupby('Month')
['total_amount'].sum().reindex(month_order)
print(monthly_revenue)
plt.figure(figsize=(20,5))
sns.barplot(x=monthly_revenue.index, y=monthly_revenue.values,
palette="dark")
plt.xlabel("Month")
plt.ylabel("Total Amount")
plt.title("Monthly Revenue Analysis")
plt.show()
```

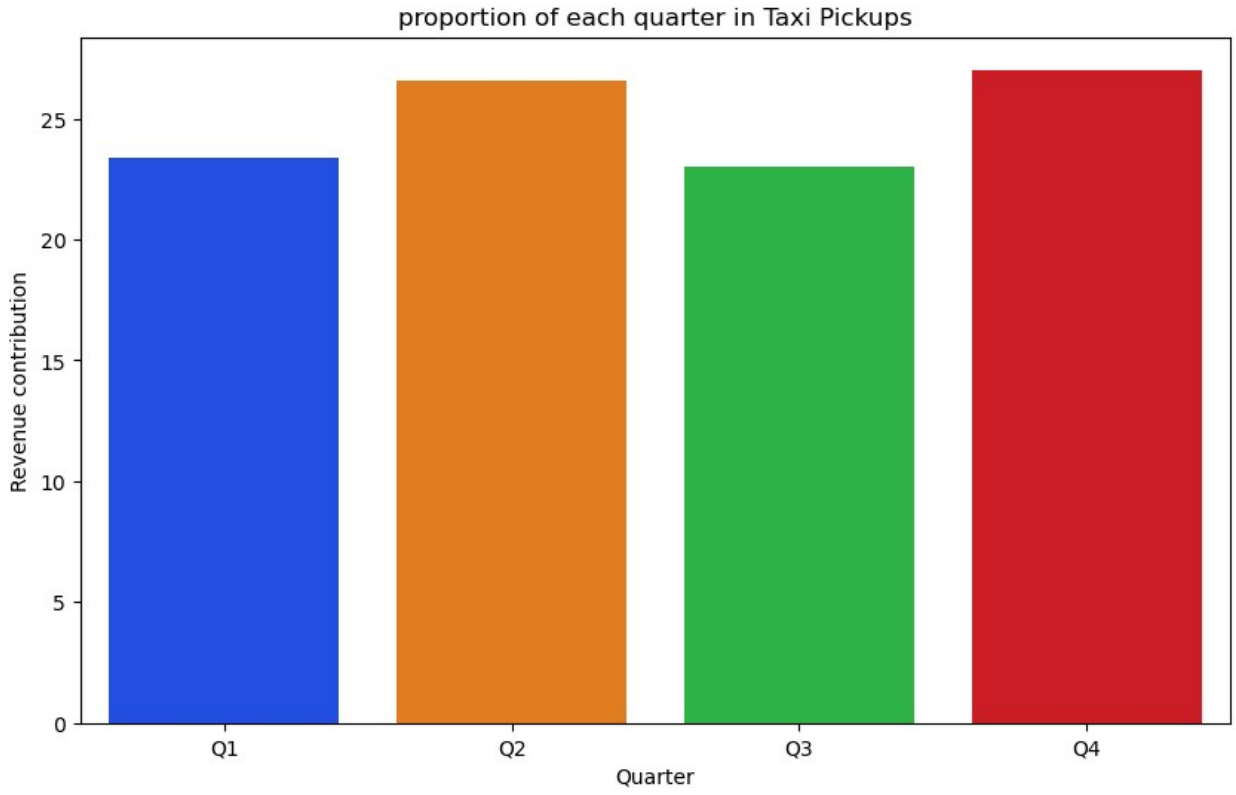
```
Month
January      4160707.47
February     3941541.05
March        4755724.14
April        4678224.05
May          5110992.82
June         4812241.75
July         4177265.69
August       4083149.71
September    4383325.85
October      5180444.37
November     4807854.06
December     4862356.04
Name: total_amount, dtype: float64
```



3.1.5 [3 marks] Show the proportion of each quarter of the year in the revenue

```
# Calculate proportion of each quarter
df['quarter'] = 'Q' +
df['tpep_pickup_datetime'].dt.quarter.astype(str)
quarter_order = ["Q1","Q2","Q3","Q4"]
quarter_revenue=df.groupby('quarter')
['total_amount'].sum().reindex(quarter_order)
print(quarter_revenue)
quarter_proportion = (quarter_revenue / quarter_revenue.sum()) * 100
print(quarter_proportion)
plt.figure(figsize=(10,6))
sns.barplot(x=quarter_proportion.index, y=quarter_proportion.values,
palette="bright")
plt.xlabel("Quarter")
plt.ylabel("Revenue contribution")
plt.title("proportion of each quarter in Taxi Pickups")
plt.show()
```

```
quarter
Q1    12857972.66
Q2    14601458.62
Q3    12643741.25
Q4    14850654.47
Name: total_amount, dtype: float64
quarter
Q1    23.397775
Q2    26.570413
Q3    23.007936
Q4    27.023877
Name: total_amount, dtype: float64
```



3.1.6 [3 marks] Visualise the relationship between `trip_distance` and `fare_amount`. Also find the correlation value for these two.

Hint: You can leave out the trips with trip_distance = 0

```
# Show how trip fare is affected by distance
```

3.1.7 [5 marks] Find and visualise the correlation between:

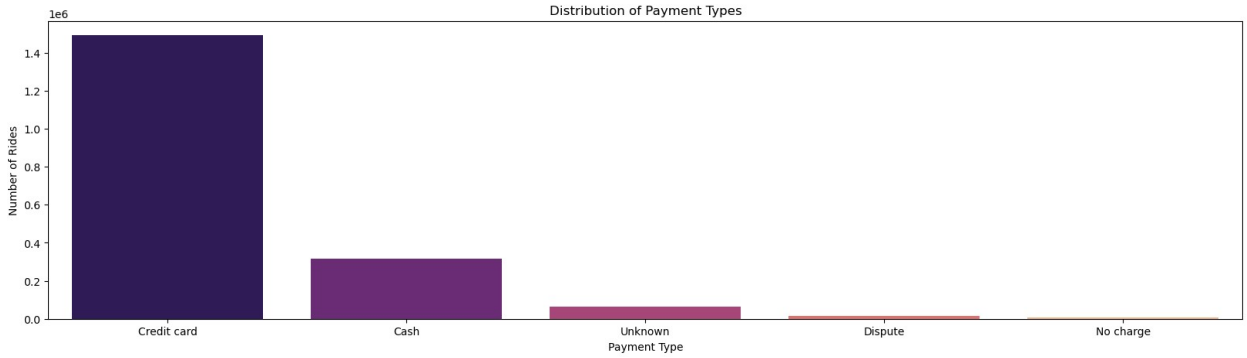
1. `fare_amount` and trip duration (pickup time to dropoff time)
2. `fare_amount` and `passenger_count`
3. `tip_amount` and `trip_distance`

```
# Show relationship between fare and trip duration

# Show relationship between fare and number of passengers

# Show relationship between tip and trip distance
```

3.1.8 [3 marks] Analyse the distribution of different payment types (`payment_type`)

```
# Analyse the distribution of different payment types (payment_type).
payment_type_labels = {
    1: 'Credit card',
    2: 'Cash',
    3: 'No charge',
    4: 'Dispute',
    5: 'Unknown'
}

df['payment_type_label'] = df['payment_type'].map(payment_type_labels)

# Count and sort
payment_counts = df['payment_type_label'].value_counts()
print(payment_counts)
# Plot
plt.figure(figsize=(20, 5))
sns.barplot(x=payment_counts.index,
y=payment_counts.values,palette="magma")
plt.title("Distribution of Payment Types")
plt.ylabel("Number of Rides")
plt.xlabel("Payment Type")
plt.show()
```

```
payment_type_label
Credit card    1492202
Cash            316189
Unknown          64831
Dispute          13570
No charge         8888
Name: count, dtype: int64
```



- 1= Credit card
- 2= Cash
- 3= No charge
- 4= Dispute

Geographical Analysis

For this, you have to use the *taxi_zones.shp* file from the *taxi_zones* folder.

There would be multiple files inside the folder (such as *.shx, .sbx, .sbn* etc). You do not need to import/read any of the files other than the shapefile, *taxi_zones.shp*.

Do not change any folder structure - all the files need to be present inside the folder for it to work.

The folder structure should look like this:

```
Taxi Zones
|- taxi_zones.shp.xml
|- taxi_zones.prj
|- taxi_zones.sbn
|- taxi_zones.shp
|- taxi_zones.dbf
|- taxi_zones.shx
|- taxi_zones.sbx
```

You only need to read the `taxi_zones.shp` file. The *shp* file will utilise the other files by itself.

We will use the *GeoPandas* library for geopgraphical analysis

```
import geopandas as gpd
```

More about geopandas and shapefiles: About

Reading the shapefile is very similar to *Pandas*. Use `gpd.read_file()` function to load the data (*taxi_zones.shp*) as a GeoDataFrame. Documentation: Reading and Writing Files

```
!pip install geopandas
```

```
Requirement already satisfied: geopandas in
/opt/anaconda3/lib/python3.12/site-packages (1.0.1)
Requirement already satisfied: numpy>=1.22 in
/opt/anaconda3/lib/python3.12/site-packages (from geopandas) (1.26.4)
Requirement already satisfied: pyogrio>=0.7.2 in
/opt/anaconda3/lib/python3.12/site-packages (from geopandas) (0.10.0)
Requirement already satisfied: packaging in
/opt/anaconda3/lib/python3.12/site-packages (from geopandas) (24.1)
Requirement already satisfied: pandas>=1.4.0 in
/opt/anaconda3/lib/python3.12/site-packages (from geopandas) (2.2.2)
Requirement already satisfied: pyproj>=3.3.0 in
/opt/anaconda3/lib/python3.12/site-packages (from geopandas) (3.7.1)
Requirement already satisfied: shapely>=2.0.0 in
/opt/anaconda3/lib/python3.12/site-packages (from geopandas) (2.1.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/anaconda3/lib/python3.12/site-packages (from pandas>=1.4.0-
>geopandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/opt/anaconda3/lib/python3.12/site-packages (from pandas>=1.4.0-
>geopandas) (2024.1)
```

```
Requirement already satisfied: tzdata>=2022.7 in
/opt/anaconda3/lib/python3.12/site-packages (from pandas>=1.4.0-
>geopandas) (2023.3)
Requirement already satisfied: certifi in
/opt/anaconda3/lib/python3.12/site-packages (from pyogrio>=0.7.2-
>geopandas) (2024.12.14)
Requirement already satisfied: six>=1.5 in
/opt/anaconda3/lib/python3.12/site-packages (from python-
dateutil>=2.8.2->pandas>=1.4.0->geopandas) (1.16.0)
```

3.1.9 [2 marks] Load the shapefile and display it.

```
import geopandas as gpd


# Read the shapefile using geopandas
zones =
gpd.read_file(r'/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/taxi_zones/taxi_zones.shp')
zones.head()

# r'/Users/milindbuwe/Desktop/Milind_Upgrad/Datasets and
Dictionary/trip_records
```

```
   OBJECTID  Shape_Leng  Shape_Area                     zone
LocationID  \
0         1    0.116357    0.000782           Newark Airport
1
1         2    0.433470    0.004866              Jamaica Bay
2
2         3    0.084341    0.000314  Allerton/Pelham Gardens
3
3         4    0.043567    0.000112             Alphabet City
4
4         5    0.092146    0.000498             Arden Heights
5

         borough                                           geometry
0            EWR  POLYGON ((933100.918 192536.086, 933091.011 19...
1         Queens  MULTIPOLYGON (((1033269.244 172126.008, 103343...
2          Bronx  POLYGON ((1026308.77 256767.698, 1026495.593 2...
3      Manhattan  POLYGON ((992073.467 203714.076, 992068.667 20...
4  Staten Island  POLYGON ((935843.31 144283.336, 936046.565 144...
```

Now, if you look at the DataFrame created, you will see columns like: `OBJECTID`,`Shape_Leng`, `Shape_Area`, `zone`, `LocationID`, `borough`, `geometry`.

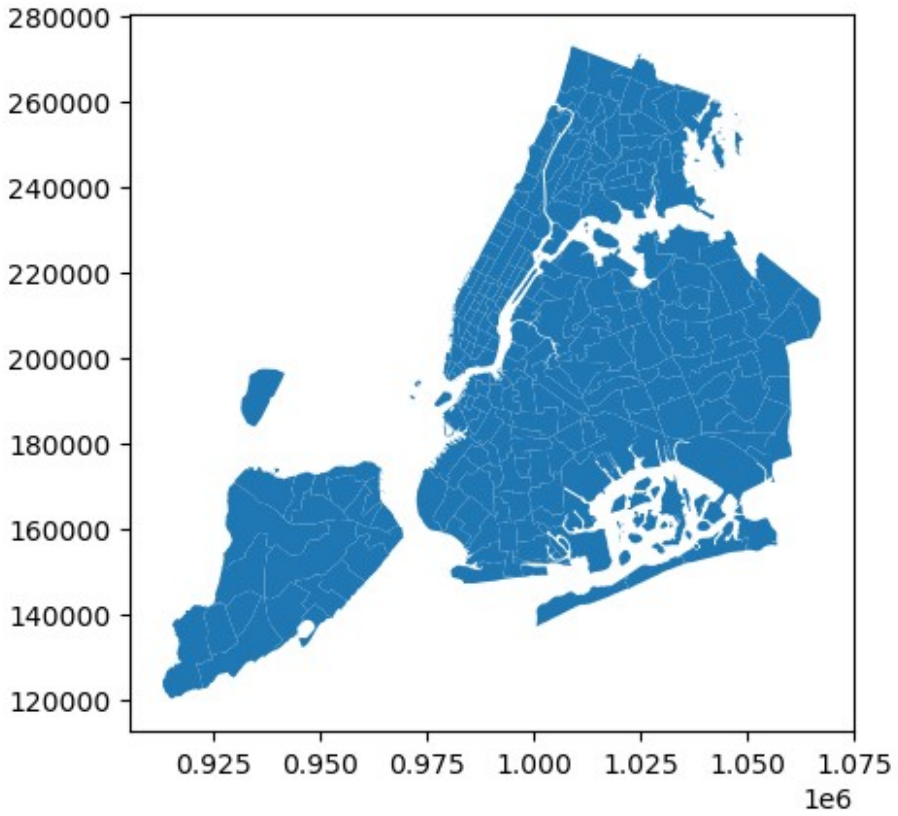
Now, the `locationID` here is also what we are using to mark pickup and drop zones in the trip records.

The geometric parameters like shape length, shape area and geometry are used to plot the zones on a map.

This can be easily done using the `plot()` method.

```
print(zones.info())
zones = gpd.GeoDataFrame(zones, geometry='geometry')
zones.plot()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 263 entries, 0 to 262
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   OBJECTID    263 non-null    int32
 1   Shape_Leng  263 non-null    float64
 2   Shape_Area  263 non-null    float64
 3   zone        263 non-null    object
 4   LocationID  263 non-null    int32
 5   borough     263 non-null    object
 6   geometry    263 non-null    geometry
dtypes: float64(2), geometry(1), int32(2), object(2)
memory usage: 12.5+ KB
None

<Axes: >
```



Now, you have to merge the trip records and zones data using the location IDs.

3.1.10 [3 marks] Merge the zones data into trip data using the `locationID` and `PULocationID` columns.

```
# Merge zones and trip records using locationID and PULocationID
df=pd.merge(left=df,right=zones, how='left', left_on='PULocationID',
right_on='LocationID')
df.head()
```

```
   VendorID tpep_pickup_datetime tpep_dropoff_datetime
passenger_count  \
0         2  2023-12-01 00:27:51   2023-12-01 00:50:12
1
1         2  2023-12-01 00:38:48   2023-12-01 01:01:55
1
2         2  2023-12-01 00:06:19   2023-12-01 00:16:57
1
3         2  2023-12-01 00:00:50   2023-12-01 00:14:37
1
4         2  2023-12-01 00:16:07   2023-12-01 00:19:17
1

   trip_distance  RatecodeID  PULocationID  DOLocationID  payment_type
\
```

```
0           3.99         1.0           148            50              1

1           4.79         1.0           231            61              5

2           1.05         1.0           161           161              1

3           2.08         1.0           137           144              5

4           0.40         1.0            68            68              1


   fare_amount  ...     Month  quarter  payment_type_label
OBJECTID  \
0        23.30  ...  December       Q4         Credit card     148.0

1        22.43  ...  December       Q4             Unknown     231.0

2        10.70  ...  December       Q4         Credit card     161.0

3        17.22  ...  December       Q4             Unknown     137.0

4         5.10  ...  December       Q4         Credit card      68.0


   Shape_Leng  Shape_Area                    zone  LocationID    borough
\
0    0.039131    0.000070       Lower East Side       148.0  Manhattan

1    0.063420    0.000167  TriBeCa/Civic Center       231.0  Manhattan

2    0.035804    0.000072        Midtown Center       161.0  Manhattan

3    0.046108    0.000116             Kips Bay       137.0  Manhattan

4    0.049337    0.000111          East Chelsea        68.0  Manhattan


                                              geometry
0  POLYGON ((988552.836 201677.665, 988387.669 20...
1  POLYGON ((981667.364 203305, 981854.109 203130...
2  POLYGON ((991081.026 214453.698, 990952.644 21...
3  POLYGON ((991954.728 209026.462, 991949.076 20...
4  POLYGON ((983690.405 209040.369, 983550.612 20...

[5 rows x 31 columns]
```

3.1.11 [3 marks] Group data by location IDs to find the total number of trips per location ID

```
# Group data by location and calculate the number of trips
trip_count =
```

```
df.groupby("LocationID").size().reset_index(name="trip_count")
print(trip_count)
```

```
     LocationID  trip_count
0           1.0         217
1           2.0           2
2           3.0          44
3           4.0        2355
4           5.0          13
..          ...         ...
250       259.0          52
251       260.0         376
252       261.0        9973
253       262.0       25435
254       263.0       36333

[255 rows x 2 columns]
```

3.1.12 [2 marks] Now, use the grouped data to add number of trips to the GeoDataFrame.

We will use this to plot a map of zones showing total trips per zone.

```
# Merge trip counts back to the zones GeoDataFrame

zones=pd.merge(left=trip_count,right=zones, how='left',
left_on='LocationID', right_on='LocationID')
zones.head()
```

```
   LocationID  trip_count  OBJECTID  Shape_Leng  Shape_Area  \
0         1.0         217         1    0.116357    0.000782
1         2.0           2         2    0.433470    0.004866
2         3.0          44         3    0.084341    0.000314
3         4.0        2355         4    0.043567    0.000112
4         5.0          13         5    0.092146    0.000498

                     zone        borough  \
0          Newark Airport            EWR
1             Jamaica Bay         Queens
2  Allerton/Pelham Gardens          Bronx
3           Alphabet City      Manhattan
4           Arden Heights  Staten Island

                                              geometry
0  POLYGON ((933100.918 192536.086, 933091.011 19...
1  MULTIPOLYGON (((1033269.244 172126.008, 103343...
2  POLYGON ((1026308.77 256767.698, 1026495.593 2...
3  POLYGON ((992073.467 203714.076, 992068.667 20...
4  POLYGON ((935843.31 144283.336, 936046.565 144...
```

The next step is creating a color map (choropleth map) showing zones by the number of trips taken.

Again, you can use the `zones.plot()` method for this. Plot Method GPD

But first, you need to define the figure and axis for the plot.

`fig, ax = plt.subplots(1, 1, figsize = (12, 10))`

This function creates a figure (fig) and a single subplot (ax)

After setting up the figure and axis, we can proceed to plot the GeoDataFrame on this axis. This is done in the next step where we use the plot method of the GeoDataFrame.

You can define the following parameters in the `zones.plot()` method:

```
column = '',
ax = ax,
legend = True,
legend_kwds = {'label': "label", 'orientation':
"<horizontal/vertical>"}
```

To display the plot, use `plt.show()`.

3.1.13 [3 marks] Plot a color-coded map showing zone-wise trips

```
# Define figure and axis

# Plot the map and display it

# can you try displaying the zones DF sorted by the number of trips?
```

Here we have completed the temporal, financial and geographical analysis on the trip records.

Compile your findings from general analysis below:

You can consider the following points:

- Busiest hours, days and months
- Trends in revenue collected
- Trends in quarterly revenue
- How fare depends on trip distance, trip duration and passenger counts
- How tip amount depends on trip distance
- Busiest zones

3.2 Detailed EDA: Insights and Strategies

[50 marks]

Having performed basic analyses for finding trends and patterns, we will now move on to some detailed analysis focussed on operational efficiency, pricing strategies, and customer experience.

Operational Efficiency

Analyze variations by time of day and location to identify bottlenecks or inefficiencies in routes

3.2.1 [3 marks] Identify slow routes by calculating the average time taken by cabs to get from one zone to another at different hours of the day.

Speed on a route *X* for hour *Y* = (*distance of the route X / average trip duration for hour Y*)

```
# Find routes which have the slowest speeds at different times of the
day
```

How does identifying high-traffic, high-demand routes help us?

3.2.2 [3 marks] Calculate the number of trips at each hour of the day and visualise them. Find the busiest hour and show the number of trips for that hour.

```
# Visualise the number of trips per hour and find the busiest hour
```

Remember, we took a fraction of trips. To find the actual number, you have to scale the number up by the sampling ratio.

3.2.3 [2 mark] Find the actual number of trips in the five busiest hours

```
# Scale up the number of trips

# Fill in the value of your sampling fraction and use that to scale up
the numbers
sample_fraction =
```

3.2.4 [3 marks] Compare hourly traffic pattern on weekdays. Also compare for weekend.

```
# Compare traffic trends for the week days and weekends
```

What can you infer from the above patterns? How will finding busy and quiet hours for each day help us?

3.2.5 [3 marks] Identify top 10 zones with high hourly pickups. Do the same for hourly dropoffs. Show pickup and dropoff trends in these zones.

```
# Find top 10 pickup and dropoff zones
```

3.2.6 [3 marks] Find the ratio of pickups and dropoffs in each zone. Display the 10 highest (pickup/drop) and 10 lowest (pickup/drop) ratios.

```
# Find the top 10 and bottom 10 pickup/dropoff ratios
```

3.2.7 [3 marks] Identify zones with high pickup and dropoff traffic during night hours (11PM to 5AM)

```
# During night hours (11pm to 5am) find the top 10 pickup and dropoff
zones
# Note that the top zones should be of night hours and not the overall
top zones
```

Now, let us find the revenue share for the night time hours and the day time hours. After this, we will move to deciding a pricing strategy.

3.2.8 [2 marks] Find the revenue share for nighttime and daytime hours.

```
# Filter for night hours (11 PM to 5 AM)
```

Pricing Strategy

3.2.9 [2 marks] For the different passenger counts, find the average fare per mile per passenger.

For instance, suppose the average fare per mile for trips with 3 passengers is 3 USD/mile, then the fare per mile per passenger will be 1 USD/mile.

```
# Analyse the fare per mile per passenger for different passenger
counts
```

3.2.10 [3 marks] Find the average fare per mile by hours of the day and by days of the week

```
# Compare the average fare per mile for different days and for
different times of the day
```

3.2.11 [3 marks] Analyse the average fare per mile for the different vendors for different hours of the day

```
# Compare fare per mile for different vendors
```

3.2.12 [5 marks] Compare the fare rates of the different vendors in a tiered fashion. Analyse the average fare per mile for distances upto 2 miles. Analyse the fare per mile for distances from 2 to 5 miles. And then for distances more than 5 miles.

```
# Defining distance tiers
```

Customer Experience and Other Factors

3.2.13 [5 marks] Analyse average tip percentages based on trip distances, passenger counts and time of pickup. What factors lead to low tip percentages?

```
#  Analyze tip percentages based on distances, passenger counts and
pickup times
```

Additional analysis [optional]: Let's try comparing cases of low tips with cases of high tips to find out if we find a clear aspect that drives up the tipping behaviours

```
# Compare trips with tip percentage < 10% to trips with tip percentage
> 25%
```

3.2.14 [3 marks] Analyse the variation of passenger count across hours and days of the week.

```
# See how passenger count varies across hours and days
```

3.2.15 [2 marks] Analyse the variation of passenger counts across zones

```
# How does passenger count vary across zones


# For a more detailed analysis, we can use the zones_with_trips
GeoDataFrame
# Create a new column for the average passenger count in each zone.
```

Find out how often surcharges/extra charges are applied to understand their prevalance

3.2.16 [5 marks] Analyse the pickup/dropoff zones or times when extra charges are applied more frequently

```
# How often is each surcharge applied?
```

4 Conclusion

[15 marks]

4.1 Final Insights and Recommendations

[15 marks]

Conclude your analyses here. Include all the outcomes you found based on the analysis.

Based on the insights, frame a concluding story explaining suitable parameters such as location, time of the day, day of the week etc. to be kept in mind while devising a strategy to meet customer demand and optimise supply.

4.1.1 [5 marks] Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies

4.1.2 [5 marks]

Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

4.1.3 [5 marks] Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.